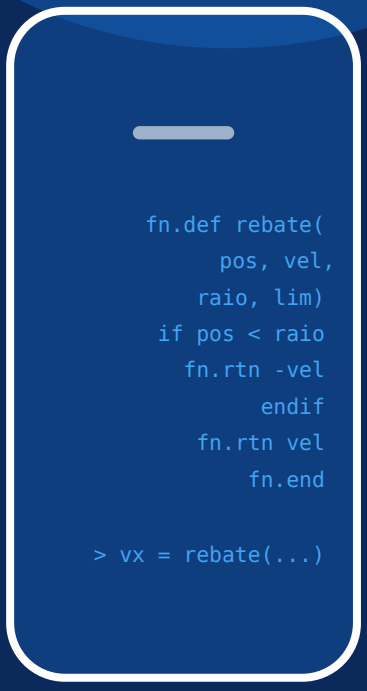
PROGRAMAÇÃO DE BOLSO RFO BASIC

AULA

07

Funções



Dê nome a pedaços de código e reaproveite-os em qualquer programa.

RECURSO DO CELULAR

Funções e include

fn.def · parâmetros · valor devolvido · escopo

Projeto de Ensino em Programação · Novembro de 2026

NESTA AULA VOCÊ VAI

- ✓ Reconhecer **repetição** no código e transformá-la em uma função.
- ✓ Criar funções com `fn.def`, `fn.rtn` e `fn.end`.
- ✓ Usar **parâmetros** e **valor devolvido**, inclusive com texto.
- ✓ Entender o **escopo**: variáveis locais e globais.
- ✓ Chamar funções **sem retorno** com `call`.
- ✓ Montar uma **biblioteca** pessoal de funções com `include`.

Olhe de novo para os programas das aulas passadas. A mesma conta de sorteio apareceu na tabuada, no dado e no duelo. Os dois blocos `if` de colisão da bolinha são quase idênticos, um para x e outro para y. Quando você copia e cola um trecho, e depois precisa corrigir o original, tem de lembrar de corrigir **todas** as cópias. Quem esquece, tem um *bug*.

A solução é dar nome a um pedaço de código e chamá-lo quando precisar. Isso é uma **função**.

1 O problema

Na Aula 3, cada pergunta sorteava dois números:

```
1  a = floor(rnd() * 9) + 1
2  b = floor(rnd() * 9) + 1
```

Duas linhas quase iguais. E se o jogo passar a sortear de 1 a 20? Você precisa trocar o `9` em dois lugares, e no duelo seriam quatro. Que tal escrever a conta uma vez só e dar a ela um nome?

2 Sua primeira função

A fim de escolher aleatoriamente um inteiro entre $[1, b]$, podemos usar a fórmula:

$$f(b) = \lfloor \mathrm{rnd}() \cdot b \rfloor + 1$$

A função possui um entrada b e uma saída $f(b)$. A parte $\lfloor \mathrm{rnd}() \cdot b \rfloor + 1$ é o corpo da função.

Transformando em código (Listagem 1):

Listagem 1: Função para realizar sorteio

`sorteia.bas` `basic`

```
1  fn.def sorteia(limite)
2    fn.rtn floor(rnd() * limite) + 1
3  fn.end
4
5  a = sorteia(9)
6  b = sorteia(20)
7  print int$(a) + " x " + int$(b) + " = " + int$(a * b)
8  end
```

RODAR Execute algumas vezes. A variável `a` fica sempre entre 1 e 9? E `b`, entre 1 e 20?

Cada parte da função tem um papel:

Parte	O que é
`fn.def`	Começa a **definição** da função.
`sorteia`	O **nome** que você escolhe.
`(limite)`	Os **parâmetros**, isto é, valores que a função recebe para trabalhar.
corpo	As instruções que a função executa.
`fn.rtn valor`	O **valor devolvido**.
`fn.end`	Termina a definição.

CONCEITO · FUNÇÃO

Uma função é uma **caixa preta** com nome: entram valores (**parâmetros**), ela trabalha e sai um valor (o **retorno**). É a mesma ideia da função matemática: para cada entrada, uma saída. Quem **chama** a função não precisa saber **como** ela calcula, só **o que** ela faz.

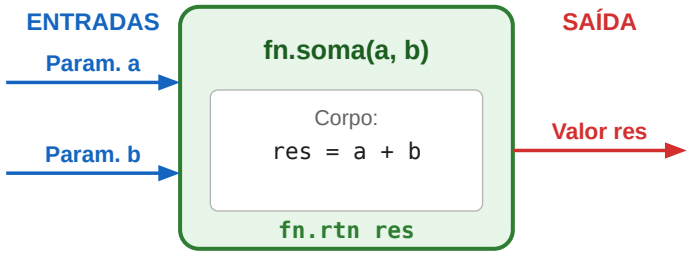


Figura 1: Representação do fluxo de uma função: as entradas (parâmetros) são processadas na caixa da função e o resultado retorna pelo comando `fn.rtn`.

ATENÇÃO

A função precisa ser **declarada antes** de ser chamada. Coloque as definições no **começo** do programa.

Agora você pode reescrever o jogo da Aula 3. Em vez de repetir a conta, chamamos `sorteia`:

Listagem 2: Quiz da tabuada

`for-quiz.bas.bas` `basic`

```
1   fn.def sorteia(limite)
2     fn.rtn floor(rnd() * limite) + 1
3   fn.end
4
5   acertos = 0
6   total = 10
7
8   for i = 1 to total
9     a = sorteia(9)
10    b = sorteia(9)
11    input int$(a) + " x " + int$(b) + " = ?", resp
12    if resp = a * b
13      acertos = acertos + 1
14    endif
15  next i
16
17  print "Você acertou " + int$(acertos) + " de " + int$(total)
18  end
```

DICA

O nome `sorteia` explica o que acontece. Ler `a = sorteia(9)` é mais fácil do que decifrar `floor(rnd() * 9) + 1`. Uma boa função também serve de **documentação** do programa.

Se o nome da função termina em `$`, como as variáveis que armazenam texto, ela devolve **texto**. Lembra da cadeia de `elseif` do nível de posição? Ela cabe numa função:

Listagem 3: Função que determina a posição do celular

`posicao.bas` `basic`

```
1   fn.def posicao$(x, y, z)
2     if z > 8
3       fn.rtn "deitado, tela para cima"
4     endif
5     if z < -8
6       fn.rtn "deitado, tela para baixo"
7     endif
8     if y > 8
9       fn.rtn "de pé"
10    endif
11    if y < -8
12      fn.rtn "de cabeça para baixo"
13    endif
14    if abs(x) > 8
15      fn.rtn "de lado"
16    endif
17    fn.rtn "inclinado"
18  fn.end
19
20  print posicao$(0, 9.7, 0.5)
```

CONCEITO · RETORNO ANTECIPADO

O `fn.rtn` encerra a função na hora. Por isso a cadeia de `if` pode ser uma lista de testes simples. Logo, assim que um deles acertar, a função devolve o resultado e para. O último `fn.rtn` é o caso “nenhuma das anteriores”.

RODAR Teste `posicao$` com `(0, 0, 9.8)`, `(0, -9.8, 0)` e `(5, 5, 5)`. Os resultados batem com o que você espera?

3 Funções sem retorno e chamada

Nem toda função precisa devolver algo. Algumas só fazem alguma coisa como falar, desenhar, escrever. Elas simplesmente não têm `fn.rtn`:

Listagem 4: Função sem retorno explícito

saudacao.bas basic

```
1   fn.def apresenta(nome$)
2     print "Olá, " + nome$ + "!"
3     tts.speak "Olá, " + nome$ + "!"
4   fn.end
5
6   tts.init
7   call apresenta("Ana")
8   apresenta("Bruno")
9   pause 3000
10  end
```

O comando `call` chama a função e ignora o que ela devolve. Uma função sem `fn.rtn` devolve `0` (ou texto vazio `""`, se o nome terminar em `$`) quando chega ao `fn.end`.

Na linha 8, chamamos a função **sem** o `call` e também funciona. O uso de `call` é mais seguro, principalmente quando o nome da função começa com uma palavra reservada da linguagem (por exemplo, um nome que comece com `print`), porque o `call` evita confusão com o comando.

> **ATENÇÃO**
>
> Quando atribuímos o valor da saída da função a uma variável, não devemos usar `call`. Ou seja, não podemos fazer `a = call sorteia(9)`.

4 Variáveis locais e globais

O que acontece com as variáveis criadas dentro de uma função? Por padrão, elas só existem dentro da função. Após o término, elas são descartadas.

Listagem 5: Variável global e local

variavel-global.bas basic

```
1  fn.def dobra(n)
2    resultado = n * 2
3    fn.rtn resultado
4  fn.end
5
6  resultado = 100
7  x = dobra(5)
8  print resultado
9  print x
```

A variável `resultado` de dentro da função é outra, embora com o mesmo nome. Dizemos que ela é **local**, existe só enquanto a função roda. A `resultado` de fora é **global**.

Tipo	Onde vive	A função enxerga?
Local	Dentro da função: parâmetros e variáveis criadas ali.	Sim, só dentro dela.
Global	No programa principal, fora das funções.	**Não**, por padrão.

CONCEITO · ESCOPO

Escopo é a região do programa onde um nome vale. A função vive numa “sala fechada”: só vê o que recebe como parâmetro (ou o que importar de propósito). Isso protege o programa, pois uma função não bagunça por acidente as variáveis de quem a chamou.

5 Passagem de argumentos

Há dois modos de passagem de argumentos: por valor e por referência. A passagem por valor já conhecemos, a variável é local, apenas uma cópia do valor é passado.

Listagem 6: Passagem por valor e por referência

`valor-referencia.bas` `basic`

```
1 fn.def incrementa(n)
2   n = n + 1
3 fn.end
4
5 resultado = 100
6 incrementa(resultado)
7 print resultado
```

Quando fazemos `incrementa(resultado)`, estamos passando o valor `100` e não a variável `resultado`.

Se quisermos modificar a variável global devemos passar sua referência. Algo como o lugar na memória onde ela está armazenada. Isso pode ser feito acrescentando `&` no início do nome do argumento.

Listagem 7: Passagem por valor e por referência

`valor-referencia.bas` `basic`

```
1 incrementa(&resultado)
2 print resultado
```

Ao rodar esse exemplo, você vai notar que a função `incrementa` realmente modificou a variável `resultado` quando passamos a referência ao invés do valor.

Modo	O que acontece

Por valor	A função recebe uma **cópia**. Mudar o parâmetro lá dentro não altera a variável de fora.
Por referência	A função recebe a **própria** variável. Mudar o parâmetro lá dentro altera a variável de fora.

Lembre-se: para passar um valor escalar (número ou texto) por referência, usamos o `&` na chamada. Dessa forma, a função altera a própria variável fora dela.

6 De volta à bolinha

Funções sem retorno, como `circulo`, brilham em gráficos, em que repetimos o mesmo desenho várias vezes. Na Aula 6, os dois `if` de colisão eram a mesma lógica aplicada a eixos diferentes. A função `rebate` resolve isso.

Listagem 8: Biblioteca de funções

`minhalib.bas` `basic`

```
1   fn.def circulo(x, y, raio, r, g, b)
2     gr.color 255, r, g, b
3     gr.circle obj, x, y, raio
4   fn.end
5
6   fn.def rebate(pos, vel, raio, limite)
7     if (pos < raio & vel < 0) | (pos > limite - raio & vel > 0)
8       fn.rtn -vel
9     endif
10    fn.rtn vel
11  fn.end
```

A função `rebate` recebe a **posição**, a **velocidade**, o **raio** e o **limite** da parede, e devolve a velocidade nova. A mesma função serve para os dois eixos.

> **CONCEITO · NÃO SE REPITA**
>
> Programadores têm uma regra de não se repitir (em inglês, **DRY**: *don't repeat yourself*). Quando duas partes do código fazem quase a mesma coisa, extraia uma função. Menos código significa menos lugares para errar.

A seguir, vamos reescrever o programa em que a bolinha fica rebatendo nas paredes. Ao invés de escrevermos as funções no mesmo arquivo, vamos importá-las usando `include`. Isso é bom caso seja do nosso interesse usar as funções em outros projetos. Além disso, torna o código mais legível.

Falando nisso, vamos também refatorar o código, ou seja, reorganizar e melhorar a estrutura interna sem alterar o seu comportamento externo. Resumindo, o programa continua a fazer exatamente a mesma coisa para o utilizador final, mas o código por trás fica mais limpo, organizado e fácil de entender. A própria DRY é um técnica de refatoração.

Listagem 9: Bolinha que rebate nas pareces

`circulo-rebate.bas` `basic`

```
1   include "minhalib.bas"
2
3   gr.open 255, 11, 42, 91
4   gr.orientation 1
5   gr.screen w, h
6
7   raio = 40
8   x = w / 2 : y = h / 3
9   vx = 12   : vy = 9
10
11  do
12    x += vx : y += vy
13    vx = rebate(x, vx, raio, w)
14    vy = rebate(y, vy, raio, h)
15
16    gr.cls
17    call circulo(x, y, raio, 245, 158, 11)
18    gr.render
19
20    gr.touch toque, tx, ty
21    pause 15
22  until toque
23
24  gr.close
```

No código refatorado, você vai percerber:

1. Uso de funções para não repetir código
2. Atribuição de múltiplas variáveis na mesma linha (usando `:`)
3. Açúcar sintático (como `x += vx`)

Compare as diferenças implementadas:

Antes	Depois (com função)
Dois blocos `if`, 6 linhas.	Duas linhas com `rebate`.
A lógica de colisão aparece duas vezes.	A lógica aparece uma vez, em `rebate`.
Corrigir um bug exige mexer em dois lugares.	Corrige-se uma vez e vale para os dois eixos.

ATENÇÃO

O `include` traz o conteúdo do arquivo para dentro do programa, no ponto em que aparece. Por isso, ponha-o no começo, antes de qualquer chamada. mantenha na biblioteca apenas definições de função, uma vez que código solto lá dentro seria executado toda vez que algum programa a incluísse.

7 Documentação

Além de escolhermos um bom nome para a função e seus parâmetros é de bom grado documentarmos o que ela faz. A maneira mais rápida e fácil é comentar a função no próprio código. Os comentários em hBasic pode ser escritos de várias maneiras particulares.

Listagem 10: Comentários em BASIC

`comentarios.bas` `basic`

```
1   REM comentário em uma única linha
2
3   ! comentário em um única linha
4
5   !!
6   Esse é um bloco de comenrários.
7   Evitarmos colocar um ponto de exclamacação para cada linha
8   !!
9
10  nota = 7.5 % comentário em um única linha após código
```

Todos os comentários são ignorados quando o programa é processado.

Os comentários na Listagem 11 indicam o que a função `distancia` faz, seus parâmetros e o que ela devolve após processar o corpo.

NOTA

Neste curso, temos evitado colocar comentários no código por concisão. Essa restrição prática é fundamental, pois no desenvolvimento *mobile*, cada caractere a menos para digitar no teclado virtual reduz erros de digitação e melhora o ritmo da aula.

Por isso, adotamos uma estratégia de código minimalista, sem comentários, transferindo toda a explicação e os detalhes dos parâmetros para o texto do material.

Listagem 11: Exemplo de função documentada

documentacao.bas basic

```
!!
================================================================
Função: distancia
Calcula a distância euclidiana entre dois pontos (x1, y1) e (x2, y2).
Parâmetros:
x1, y1 - Coordenadas do Ponto A
x2, y2 - Coordenadas do Ponto B
Retorno: Distância direta em pixels (número real).
 ================================================================
!!

fn.def distancia(x1, y1, x2, y2)
  ! Aplicação da fórmula da distância (Pitágoras)
  fn.rtn sqr((x2 - x1)^2 + (y2 - y1)^2)
fn.end
```

8 Pêndulo simples

O pêndulo simples é um sistema físico composto por um corpo de massa m suspenso por um fio de comprimento L que oscila em torno de um ponto fixo de equilíbrio.

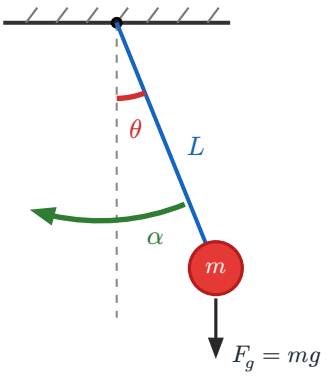


Figura 2: Esquema do pêndulo simples.

Onde

- L: comprimento da haste
- θ: ângulo entre a haste e a posição vertical de equilíbrio.
- α: aceleração angular
- m: massa do corpo
- g: constante da gravidade

Para a simulação de um pêndulo simples com a aproximação de pequenos ângulos, a aceleração angular é dada por:

$$\alpha = -\frac{g}{L} \cdot \theta$$

Em hBasic, teremos

```
1  fn.def accel(th, g, L)
2    fn.rtn -(g / L) * th
3  fn.end
```

Para calcular as coordenadas horizontal e vertical:

$$x = L \cdot \sin\theta \qquad y = L \cdot \cos\theta$$

Em hBasic, teremos

```
1  fn.def get_x(cx, L, th)
2    fn.rtn cx + L * sin(th)
3  fn.end
4
5  fn.def get_y(cy, L, th)
6     fn.rtn cy + L * cos(th)
7  fn.end
```

Nota-se que a tradução da fórmula matemática para código é bastante direta. Porém, de onde vieram esses `cx` e `cy`? Lembre-se que, na tela do dispositivo Android, a origem $(0, 0)$ fica no canto superior esquerdo. Por isso, é necessário realizar uma translação para o sistema de coordenadas do dispositivo. O código acima ajusta a posição do pêndulo considerando o ponto de apoio centrado em `(cx, cy)`.

Iremos adicionar essas funções, juntamente com uma função para desenhar a haste, a nossa biblioteca Listagem 8. Portanto, agora podemos calcular a aceleração angular α. Com α, calcularemos a velocidade angular ω. E com ω, calcularemos a posição angular θ,

$$\omega = \omega + \alpha \cdot \Delta t \qquad \theta = \theta + \omega \cdot \Delta t$$

No código do pêndulo simples, Listagem 12, criamos as duas variáveis `om` e `dt`.

1. A aceleração (`al`) altera a velocidade: `om = om + al * dt`.
2. A velocidade (`om`) altera a posição/ângulo: `th = th + om * dt`.
3. O passo de tempo (`dt`) controla o ritmo dessa atualização.

DICA

Os cálculos da velocidade e posição fazem parte da **Integração de Euler**, a técnica usada para atualizar o movimento físico quadro a quadro. Ela permite transformar as leis da Física em um algoritmo que atualiza a posição e a velocidade de um objeto ao longo do tempo e isso é exatamente o que precisamos para criar uma animação, um jogo ou uma simulação física.

Listagem 12: Biblioteca de funções

pendulo.bas basic

```
include "minhalib.bas"

gr.open 255, 11, 42, 91
gr.orientation 1
gr.screen w, h

L = 250 : g = 9.81 : th = 0.2
dt = 0.3 : om = 0
cx = w / 2 : cy = 150
raio = 20

do
  al = accel(th, g, L)
  om = om + al * dt
  th = th + om * dt

  px = get_x(cx, L, th)
  py = get_y(cy, L, th)

  gr.cls
  call linha(cx, cy, px, py, 100, 100, 100)
  call circulo(px, py, raio, 245, 158, 11)
  gr.render
  pause 16
until 0
end
```

Quebre o código!

Mude uma coisa de cada vez no código de Listagem 12 e observe o que acontece. Antes de executar, escreva a sua previsão; depois compare.

QUEBRAR Troque `th = 0.2` por `th = 0`. O que acontece? Por que o pêndulo não se mexe? (Pense: qual é a aceleração e qual é a velocidade inicial?)

QUEBRAR Troque a ordem das linhas `om = om + al * dt` e `th = th + om * dt`. Deixe rodar por alguns segundos e observe a amplitude da oscilação. O que muda?

QUEBRAR Mude o passo de tempo: `dt = 0.05`, depois `dt = 3` e depois `dt = 12`. O que cada valor faz com a velocidade da animação? Algum deles faz o pêndulo "explodir"?

QUEBRAR Em `accel`, apague o sinal de menos. Em vez de oscilar, o que o pêndulo faz? Por que o sinal importa?

QUEBRAR Em `get_x` e `get_y`, troque `sin` por `cos` e vice-versa. Para onde vai a bolinha quando `th = 0`? O que `th = 0` deve significar, fisicamente?

QUEBRAR Troque `cy + L * cos(th)` por `cy - L * cos(th)`. Onde a bolinha aparece? Que valor de `cy` a traz de volta para a tela? O que isso revela sobre a origem no canto superior esquerdo?

QUEBRAR Apague o `gr.cls` do laço. O que acontece com as linhas na tela? E com a velocidade do programa depois de algum tempo?

QUEBRAR Comece com `th = 1.5` e troque a aceleração por `fn.rtn -(g / L) * sin(th)`. O período da oscilação aumenta ou diminui em relação à versão `-(g / L) * th`? Por quê?

DICA

O segundo experimento mostra que a ordem das atualizações faz parte do método. No código original, a velocidade é atualizada primeiro, e a posição usa a velocidade nova.

Com as linhas trocadas, a posição usa a velocidade antiga (é o **Euler explícito**), e a energia do pêndulo cresce um pouco a cada passo, até a oscilação ficar maior do que devia. O primeiro esquema (**Euler semi-implícito** ou **simplético**) mantém a energia aproximadamente constante, e por isso é o preferido em jogos e simulações de física.

DESAFIO

Pêndulo com atrito

Mude o programa do pêndulo simples para comportar o componente de atrito.

1. Acrescente uma variável `b` e use `al = accel(th, g, L) - b * om`. Teste `b = 0.05`, `b = 0.4` e `b = 1`. Em qual deles o pêndulo ainda oscila? Em qual ele só "desce devagar" até parar? (Pista: o valor crítico é $b = 2\sqrt{\frac{g}{L}}$.)

Bônus: Rode dois pêndulos ao mesmo tempo, com o mesmo ângulo inicial e cores diferentes: um com a ordem correta e outro com a ordem trocada. Algum código se repete? Quais funções você criaria para evitar isso?

9 Resumo

Ideia ou comando	Para que serve
`fn.def nome(p1, p2)`	Definição de uma função que retorna um decimal.

`fn.def nome$(p1, p2)`	Definição de uma função que retorna uma *string*.
`fn.rtn valor`	Devolve um valor e **encerra** a função.
`fn.end`	Termina a definição. Sem `fn.rtn`, devolve `0` ou `""`.
`call nome(...)`	Chama a função **ignorando** o retorno.
Variável local	Existe só dentro da função (parâmetros incluídos).
`include "arquivo.bas"`	Traz as funções de um arquivo-biblioteca para o programa.

NA PRÓXIMA AULA

Aula 08: Bolha de nível. Hora de pôr as funções para trabalhar! Juntaremos o acelerômetro da Aula 05, a bolinha da Aula 06 e as funções de hoje.